Technical Report: Open-Source 5G Testbed with Over-the-Top SIP Voice & AI-Based QoS Analysis

Sharan Deepak Thakur

July 17, 2026

1 Overview

This report documents a minimal, reproducible 5G Standalone (SA) testbed built with **Open5GS** [1] for the 5G Core Network and **UERANSIM** [2] for the simulated next-generation Node B (gNB) and two User Equipments (UEs), carrying an over-the-top Session Initiation Protocol (SIP) voice service on top of the 5G user plane. The SIP service uses **Kamailio** [3] as the registrar/proxy, **RTPengine** [4] as the media relay, and two **baresip** [5] clients as UEs 1001/1002. An AI/analysis component predicts and cross-validates call quality via the Mean Opinion Score (MOS) from Real-time Transport Protocol (RTP) statistics and performs unsupervised clustering of N3 (GPRS Tunnelling Protocol User Plane, GTP-U) user-plane traffic, addressing all four functional requirements of the assessment brief. All configs and logs are available on GitHub [1]. The rest of the report is structured as follows: Section 2 describes the architecture of the testbed, Section 3 describes the 5G network setup, Section 4 describes the SIP-based voice service, Section 5 describes the AI component, and Section 6 concludes the report.

2 Architecture

The full architecture is shown in Figure 1.

```
UE-1001 / UE-1002 (UERANSIM + baresip)
        | N1 / N2 / N3
        v
    Open5GS (AMF/SMF/UPF) -- ogstun 10.45.0.0/16
        |
        v
    Kamailio :5060 <--> RTPengine :2223
        |
    SIP + RTP
```

Figure 1: Architecture of the 5G SA testbed.

Open5GS [1] provides all core network functions, including the Access and Mobility Management Function (AMF), and User Plane Function (UPF)[2], as separate processes. UERANSIM [2] simulates a single gNB and two UEs, each provisioned as a subscriber in the core's subscriber database.

3 5G Network Setup

Component	Role	Address
Kamailio	SIP registrar / proxy	`10.45.0.1:5060`
RTPengine	Media relay	`127.0.0.1:2223`
UE 1001 (baresip)	SIP client	`10.45.0.10:5070`
UE 1002 (baresip)	SIP client	`10.45.0.11:5071`

Table 1: Deployed components and their addresses.

Both UEs attach successfully. Logs from the AMF, SMF and UPF network functions record registration, authentication and Packet Data Unit (PDU) session establishment for both International Mobile Subscriber Identities (IMSIs), confirming the required minimum of two attached UEs.

[1] https://github.com/c2p-cmd/5g_assessment

[2] The remaining Open5GS network functions, are standard 3GPP core functions not discussed further in this report.

IP connectivity through the core. On successful PDU session establishment, the UPF assigns each UE an address from the user-plane TUN interface subnet `10.45.0.0/16`: UE 1001 receives `10.45.0.10` and UE 1002 receives `10.45.0.11`. This interface is the UE-side endpoint of the N3/N6 user plane, so any traffic sent from a UE's assigned address is, by construction, routed through the 5G data plane demonstrated in Figure 2.

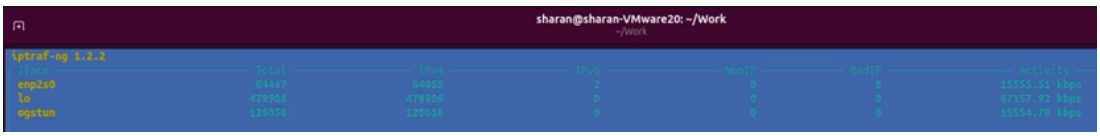


Figure 2: `iptraf` output showing packet flow through the core.

SIP over the 5G data plane. All SIP and RTP traffic in this testbed was captured directly on that user-plane interface, which is the same interface UE traffic egresses through after leaving the UPF. Kamailio and RTPengine are reachable only via this subnet (`10.45.0.1`), so the SIP clients have no path to the SIP server other than through the 5G core. This satisfies the requirement that SIP ride the 5G data plane without relying on Voice over LTE (VoLTE) or Voice over New Radio (VoNR).

4 SIP-Based Voice Service

Kamailio [3] is handling the `REGISTER` and call routing between the two baresip [5] clients. RTPengine [4] is inserted into the Session Description Protocol (SDP) as a media relay so that RTP never flows directly between UEs. Because Kamailio does not log individual SIP messages to the system journal by default, the signaling trace below was reconstructed with a protocol analyser from a full SIP/RTP traffic capture, filtering on the SIP protocol.

Registration. Both UEs register successfully:

```
18:05:43.620 10.45.0.10 -> 10.45.0.1 REGISTER (Call-ID 46c8...) 1001 -> 1001
18:05:43.621 10.45.0.1 -> 10.45.0.10 200 OK (Call-ID 46c8...) 1001 -> 1001
18:05:46.203 10.45.0.11 -> 10.45.0.1 REGISTER (Call-ID bdf5...) 1002 -> 1002
18:05:46.203 10.45.0.1 -> 10.45.0.11 200 OK (Call-ID bdf5...) 1002 -> 1002
```

Call setup and teardown. Figure 3 shows a representative call between extensions 1002 and 1001: the full expected flow, `INVITE` → `100 Trying` → `180 Ringing` → `200 OK` → `ACK`, later closed with `BYE`:

```
18:05:49.057 10.45.0.11 -> 10.45.0.1 INVITE (909f...) 1002 -> 1001
18:05:49.059 10.45.0.1 -> 10.45.0.10 INVITE (909f...) 1002 -> 1001
18:05:49.062 10.45.0.10 -> 10.45.0.1 180 (909f...)
18:05:49.079 10.45.0.10 -> 10.45.0.1 200 (909f...)
18:05:49.081 10.45.0.11 -> 10.45.0.1 ACK (909f...) 1002 -> 1001
18:06:00.319 10.45.0.11 -> 10.45.0.1 BYE (909f...) 1002 -> 1001
```

Figure 3: Call setup and teardown.

Across the experiment conducted for the analysis, four calls of this kind were placed between the two clients, plus one signaling anomaly recorded in an earlier baseline experiment: a `487`/`CANCEL` on a call that was hung up before it was answered. This is used as an example of failed-signaling behaviour rather than a defect in the setup.

Media relay. RTPengine's control-daemon log confirms it is actively brokering each call's SDP and RTP path. It receives `offer`/`answer` commands from Kamailio, allocates relay ports, and confirms the learned peer address once RTP starts flowing:

```
18:03:59 [8ab7674b...]: Received command 'offer' from 127.0.0.1:53322
18:03:59 [8ab7674b...]: Received command 'answer' from 127.0.0.1:53322
18:03:59 [8ab7674b.../.../1 port 30059]: Confirmed peer address as 10.45.0.11:18147
18:03:59 [8ab7674b.../.../1 port 30024]: Confirmed peer address as 10.45.0.10:9964
18:04:03 [8ab7674b...]: Received command 'delete' from 127.0.0.1:53322
```

Figure 4 shows the same call flow visualised in `sngrep`, which was used during development to sanity-check the signaling live.

5 AI Component: QoS Prediction and Traffic Analysis

The AI component combines an analytical Quality of Service (QoS) model with unsupervised traffic classification, working from three independent evidence sources for the same calls: RTPengine's own statistics, an independently re-computed E-model score, and a ground-truth RTP parse straight from the packet capture (PCAP).

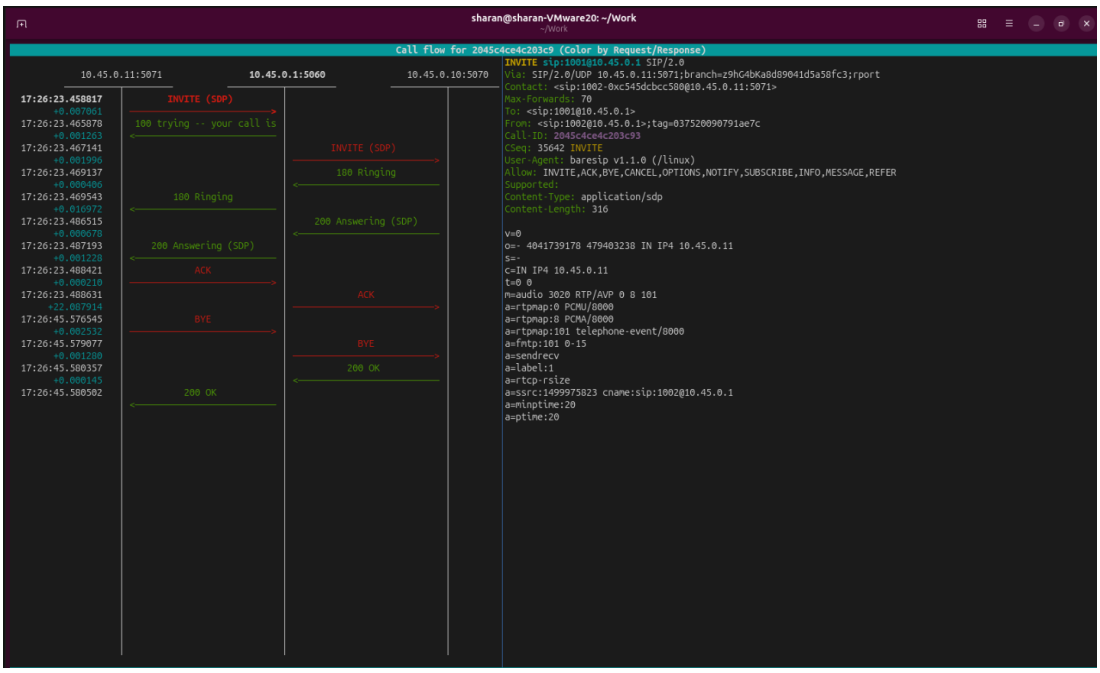


Figure 4: `sngrep` view of SIP signaling captured on the 5G user-plane interface (REGISTER, INVITE, 180, 200, ACK, BYE).

5.1 Measured call quality (RTPengine)

RTPengine emits per-leg *Final packet stats* on session teardown. Parsing these statistics over the four calls in Experiment 2 yields 10 RTP legs (2 per call: UE↔core in each direction), all using PCMU/8000. RTPengine reports **0% packet loss** and **MOS 4.3** on every leg, with per-leg jitter between 0.0–2.0 ms and end-to-end round-trip time (RTT) between 2.1–4.5 ms, consistent with a lightly loaded, single-host testbed environment.

5.2 Independent MOS via the ITU-T G.107 E-model

To validate RTPengine's figure rather than simply trust it, MOS is recomputed from the same raw inputs (jitter, loss, RTT) using the International Telecommunication Union Telecommunication Standardization Sector (ITU-T) G.107 E-model [6] for the PCMU/G.711 codec ($I_e = 0$, $B_{pl} = 4.3$):

$$R = 93.2 - I_d - I_{e,\text{eff}} \tag{1}$$

$$\text{MOS} = f(R) \tag{2}$$

where I_d is the delay impairment (from one-way delay $\approx$ RTT/2 plus jitter buffer) and $I_{e,\text{eff}}$ is the effective equipment/loss impairment. Applied to all 10 legs, this yields MOS values in 4.407 to 4.409, against RTPengine's reported 4.3. The two estimates track each other closely: 0.108 MOS points with a maximum mean absolute error of 0.109. Figure 5 plots the two estimates against each other. The small, consistent offset (RTPengine's engine rounds/clamps differently at the high-MOS end) is far smaller than the ± 0.5 resolution typically considered meaningful for MOS, so the independent model is considered a successful cross-validation of RTPengine's measurement.

5.3 Ground-truth cross-check from raw RTP

For one representative call, one leg was independently re-derived straight from the packet capture: packet loss from RTP sequence-number gaps, jitter from inter-arrival-time variance. **Table 2** compares all three sources for the same leg.

Metric	PCAP (ground truth)	RTPengine	E-model (from PCAP metrics)
Packets	1193	—	—
Loss	0.000%	0.000%	—
Jitter	0.31 ms	1.00 ms	—
MOS	—	4.3	4.408

Table 2: Cross-check of leg `10.45.0.10:16084 <> 10.45.0.1:30000` from three independent sources.

All three sources agree on zero loss and a MOS in the "excellent" band, with jitter estimates in the same low-millisecond regime (RTPengine's coarser 1 ms bucket vs. the raw 0.31 ms computed from packet timestamps).

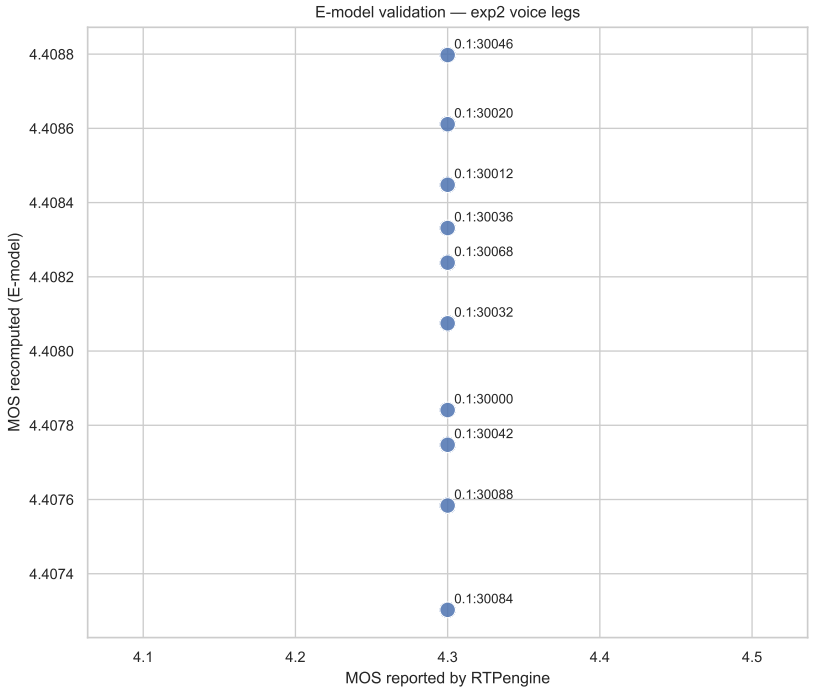


Figure 5: RTPengine-reported MOS vs. independently recomputed E-model MOS, per RTP leg.

This cross-check also surfaced a topology detail worth flagging: this leg is *not* a direct UE-to-UE RTP stream. `10.45.0.10` is UE1 and `10.45.0.1` is the core-side RTPengine relay; the call's other leg (`10.45.0.1:30068 <> 10.45.0.11:40196`) carries the media onward to UE2. RTPengine anchors media in the middle of the call, so every call is actually two independent UE↔core legs rather than one end-to-end UE↔UE leg, which matters when interpreting per-leg statistics. Figure 6 shows the inter-arrival-time distribution used for this jitter estimate.

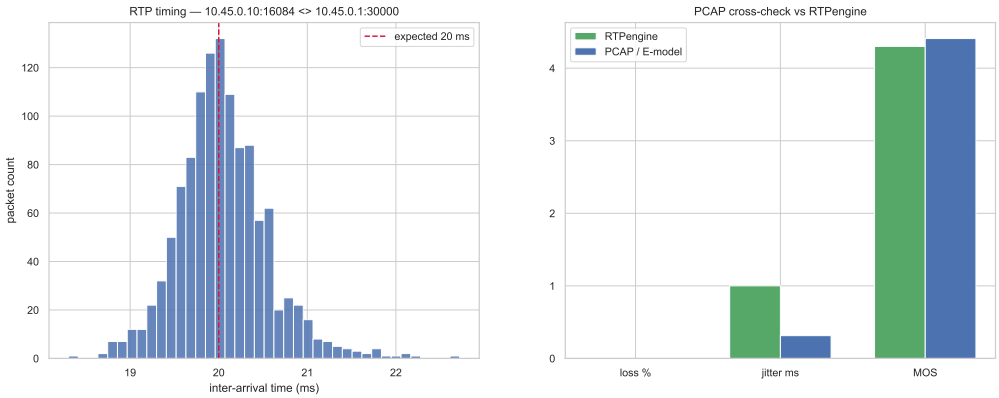


Figure 6: RTP inter-arrival-time histogram and derived metrics for the cross-checked leg.

5.4 Unsupervised N3 traffic clustering

Beyond MOS validation, GTP-U user-plane traffic on N3 (342 flows total after decapsulation, pooled across both experiments) was clustered *without using any SIP header information*, to test whether QoS-relevant traffic classes are separable from flow-level features alone (packet count, mean size, packets/sec, inter-arrival time). Features were standardised and K-Means [7, 9] was swept from $k = 2$ to $k = 11$, tracking inertia and silhouette score [8] (Figure 7).

Silhouette score peaks around $k = 7$, but at that resolution clusters appeared to over-fit statistical noise rather than reflect genuinely distinct traffic behaviour. The inertia curve elbows at $k = 4$; $k = 3$ is arguably the more conservative, "natural" choice, but $k = 4$ was used for the final clustering as it separates bulk downloads into two sub-clusters (likely single- vs. parallel-stream downloads) while keeping VoIP and signaling each in one cluster. Comparing clusters against ground-truth flow labels (derived from port/protocol at capture time) shows bulk TCP/HTTPS flows separate cleanly (121/122 in the two bulk-download clusters), and SIP signaling is isolated correctly. DNS queries, however, are consistently grouped into the same cluster as VoIP/RTP traffic (157 DNS flows alongside 10 RTP-voice flows) rather

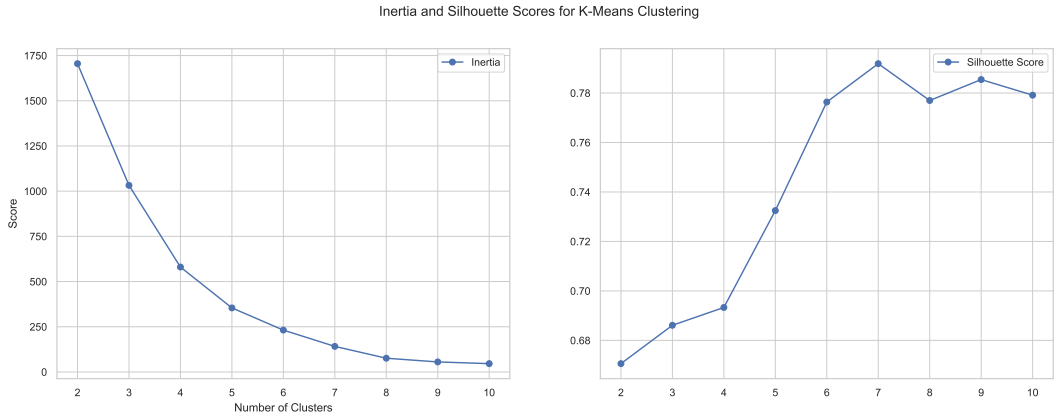


Figure 7: Inertia (elbow) and silhouette score vs. number of K-Means clusters, $k = 2$ to $k = 11$.

than with SIP signaling. Both DNS and RTP produce small, frequent, low-inter-arrival-time UDP packets, so at the flow-feature level (without payload inspection) they are statistically closer to each other than to bursty SIP request/response traffic. This is a genuine limitation of pure flow-based clustering for this use case, and a natural next step would be to add payload-derived features (e.g. explicit port awareness or deep packet inspection, DPI) to disambiguate DNS from RTP. Figure 8 projects the four discovered clusters onto their top three principal components (PCA [9]), visualising how cleanly bulk-download traffic separates from the VoIP/DNS/signaling group.

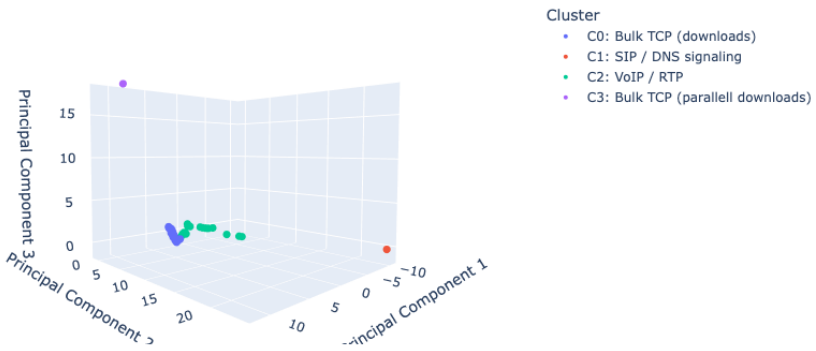


Figure 8: 3D PCA projection of the $k = 4$ K-Means clusters over combined N3 flow features from both experiments.

6 Conclusion

The implementation meets all four functional requirements of the exercise: two UEs attach to a 5G core with working IP connectivity as well as SIP registration, call setup and teardown succeed over that data plane via Kamailio. The flow demonstrated end-to-end over the 5G user plane and in logs. The AI component goes beyond a single script by validating RTPengine's MOS against an independently implemented E-model and raw-packet ground truth, and by attempting unsupervised traffic classification on the N3 user plane.

References

[1] Open5GS: *Open Source implementation for 5G Core and EPC*. https://open5gs.org (accessed 17th July 2026).

[2] A. Güngör, *UERANSIM: Open source 5G UE and RAN (gNodeB) simulator*. https://github.com/aligungr/UERANSIM (accessed 17th July 2026).

[3] Kamailio SIP Server Project: *Kamailio – the Open Source SIP Server*. https://www.kamailio.org (accessed 17th July 2026).

[4] Sipwise GmbH, *rtpengine: NGCP RTP/media Proxy*. https://github.com/sipwise/rtpengine (accessed 17th July 2026).

[5] A. Casajus Ramo et al., *baresip: A modular SIP User-Agent*. https://github.com/baresip/baresip (accessed 17th July 2026).

[6] ITU-T Recommendation G.107, *The E-model: a computational model for use in transmission planning*. International Telecommunication Union, 2015.

[7] S. Lloyd, "Least squares quantization in PCM," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[8] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

[9] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.